Lab 9: Image Priors and Compressive Imaging

Due Wednesday, April 15, 11:59pm

In Lab 8, you implemented FISTA and explored non-negativity and native sparsity priors. In this lab, we compare additional image priors—wavelet sparsity and total variation—and see their effect on reconstruction quality. We will also explore compressive imaging, where we recover images from fewer measurements than unknowns.

Each prior is implemented as a proximal operator that you pass to `InverseSolver` via the `prox_fn` argument, just like `prox_native` in Lab 8. By comparing these different priors, you will be using FISTA to solve the following optimization problems:

$$\text{non-negativity prior:} \quad L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \text{non-neg}(\mathbf{x}) \tag{1}$$

$$\text{native prior:} \quad L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|\mathbf{x}\|_1 \tag{2}$$

$$\text{wavelet sparsity:} \quad L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|W(\mathbf{x})\|_1 \tag{3}$$

$$\text{TV prior:} \quad L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|\nabla\mathbf{x}\|_1 \tag{4}$$

Allowed libraries. You may use `numpy`, `scipy`, `matplotlib`, `imageio`, and `pywt` (PyWavelets).

Starter code. Use your completed `inverse_solver.py` from Lab 7 and `priors.py` from Lab 8. You will add two new proximal operators to `priors.py`: `prox_wavelet` and `prox_tv`.

Each proximal operator follows the signature `prox_fn(x,lam,step_size)` and can be passed directly to `InverseSolver`:

```
from inverse_solver import InverseSolver
from priors import prox_tv

solver = InverseSolver(psf, algorithm='fista',
                       prox_fn=prox_tv, lam=1e-4)
x_hat = solver(y)
```

Your data. Use your own DiffuserCam images for all problems. You may use the provided sample data for debugging, but your submission must show results on your own capture.

1 Wavelet Sparsity

Native sparsity may be a good prior for something that's mostly black with a few bright points (e.g. stars), but it will not work well for natural images. Instead, we can try to use sparsity in another domain. One common sparsity domain is wavelets.

Wavelets decompose an image into components at different scales and orientations. The coarsest level captures the overall structure (e.g. the shape of an object), while finer levels capture progressively smaller details (e.g. edges, textures). Natural images tend to have most of their energy concentrated in the coarse-scale coefficients, with relatively few large fine-scale coefficients. This makes them approximately sparse in the wavelet domain: most wavelet coefficients are near zero, and only a small fraction are needed to represent the image well.

The loss function with a wavelet prior becomes:

$$L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|W(\mathbf{x})\|_1$$

where $W(\cdot)$ is the wavelet transform. Rather than soft-thresholding the image in the native basis, you will now soft-threshold the image in the wavelet basis, and do an inverse wavelet transform to bring the image back to the native domain. Thus, the proximal update step becomes:

$$\mathbf{x}_{k+1} \leftarrow W^T \,\text{soft}(W(\mathbf{w}_k),\, \lambda/L)$$

where $W(\cdot)$ and $W^T(\cdot)$ are the forward and inverse wavelet transforms. The functions `W` and `WT` are already provided in `priors.py` (using Daubechies-4 wavelets with 4 levels of decomposition).

To see what these functions do, you can run `Wx,coeffs=W(x)` and plot `Wx` to visualize the wavelet transform. When you do the inverse wavelet transform, you should get back exactly your original image, since $\mathbf{x} = W^T(W(\mathbf{x}))$. Try soft-thresholding the wavelet coefficients before applying the inverse wavelet transform. What happens as you increase λ and remove more of the wavelet coefficients?

Answer the following questions, and then implement the wavelet prior and try running FISTA with wavelet sparsity. Make sure to tune λ until you see an effect. Make sure to plot the loss ($L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|W(\mathbf{x})\|_1$) over iteration.

Recall that `W` and `WT` are the forward and inverse wavelet transforms:

```
Wx, slices = W(x_hat) # forward wavelet transform
x_recon = WT(Wx, slices) # inverse (should match x_hat)
```

Hint: to zero out the smallest coefficients, you will need to find a magnitude threshold such that only the top $K\%$ survive, then set the rest to zero before calling `WT`.

Problem 1.1:

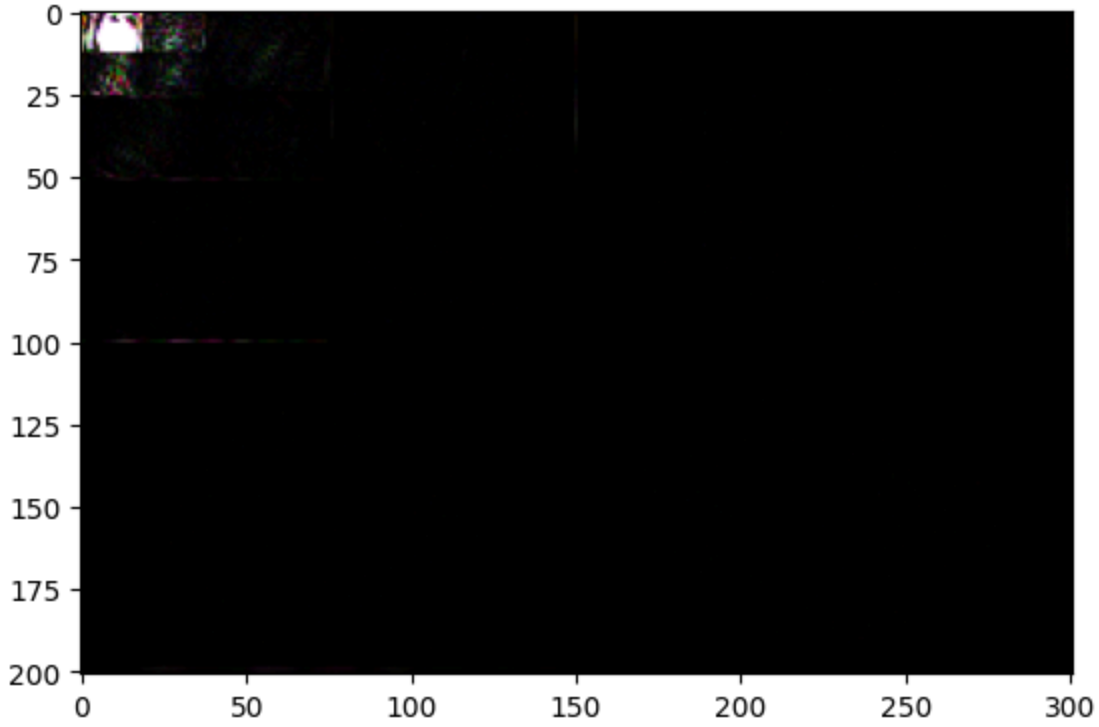
(a) Plot the wavelet coefficient image using `Wx,slices=W(x_hat)` and `imshow` (use a log scale for visibility: `np.log1p(np.abs(Wx))`). Then plot a histogram of your reconstructed image's pixel magnitudes and a histogram of the wavelet coefficient magnitudes. Use a logarithmic y-axis for both histograms so you can see the tails. How does the sparsity of the image differ across the two domains?

(b) Zero out the smallest wavelet coefficients, keeping only the top $K\%$ by magnitude, and reconstruct with `WT()`. Start at 50% and decrease until the PSNR drops below 22 dB. For each retention level, compute the PSNR relative to the original image and report the number of non-zero coefficients remaining. Display the reconstructions side by side. How few coefficients can you keep before the image becomes unrecognizable?

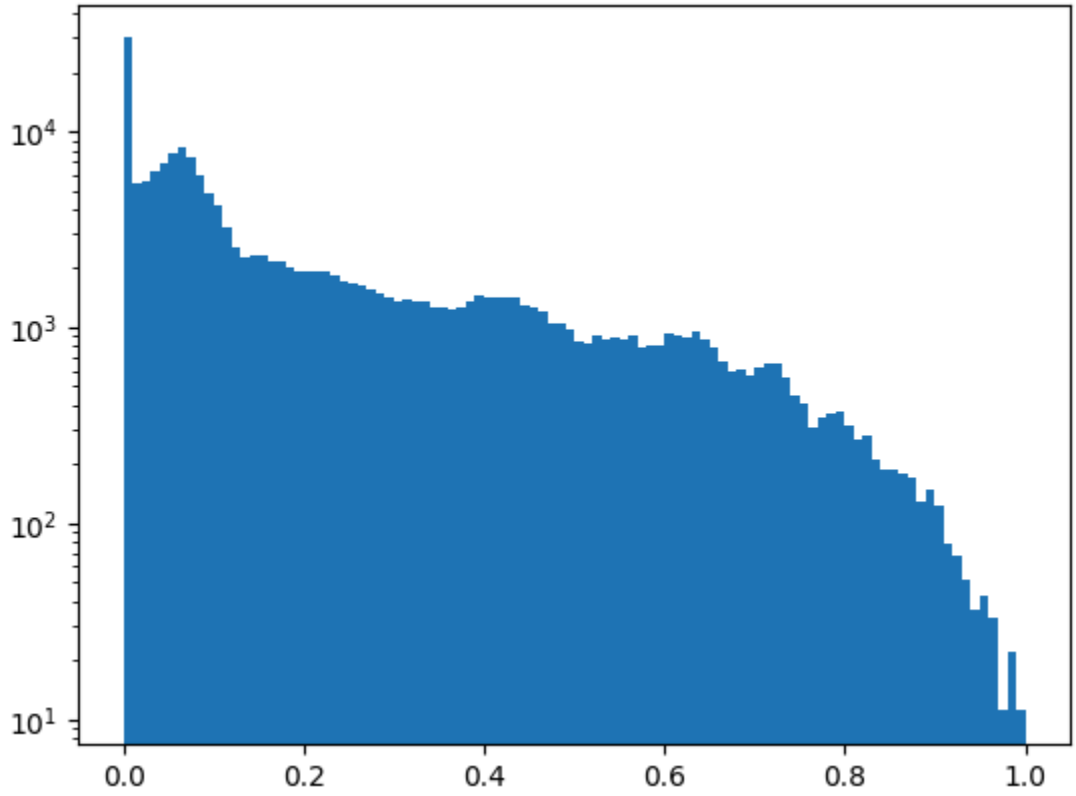
(c) Fill in `prox_wavelet` in `priors.py`. Then run FISTA with `prox_fn=prox_wavelet`, sweeping λ on your DiffuserCam image for 200 iterations. Display reconstructions for each.

(d) What happens as λ increases? Compare your best wavelet result to the native sparsity result from Lab 8. Which prior better preserves image detail?

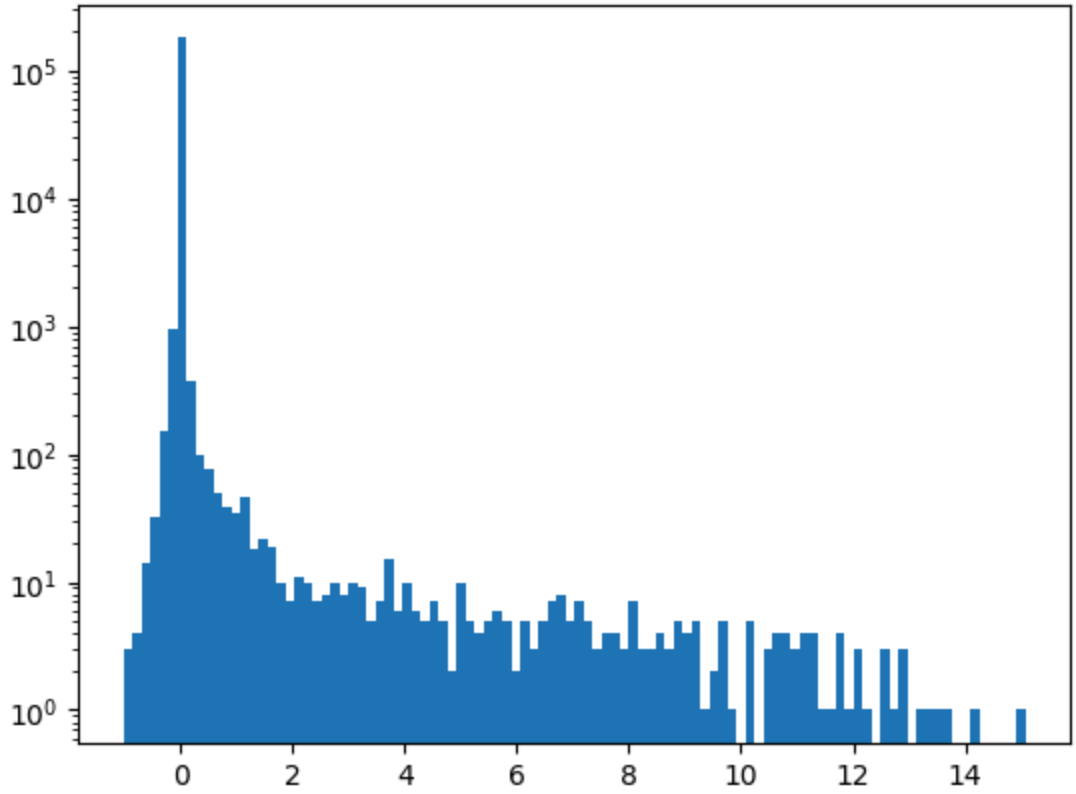
a)



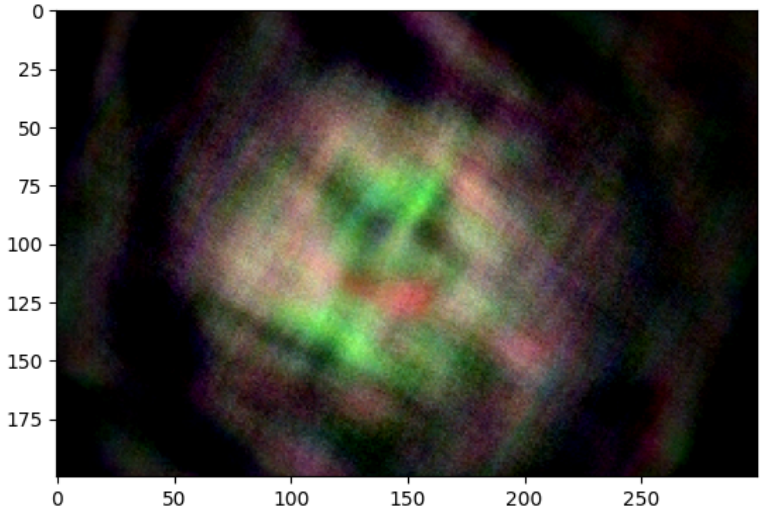
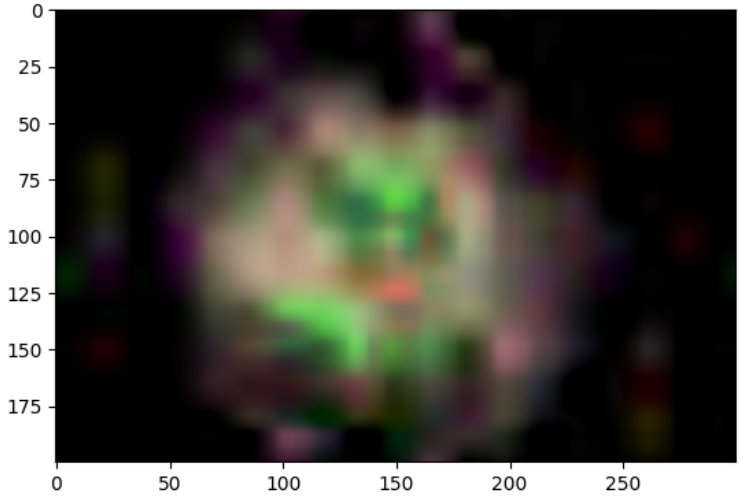
Reconstructed Image Histogram:



Wavelet Histogram:



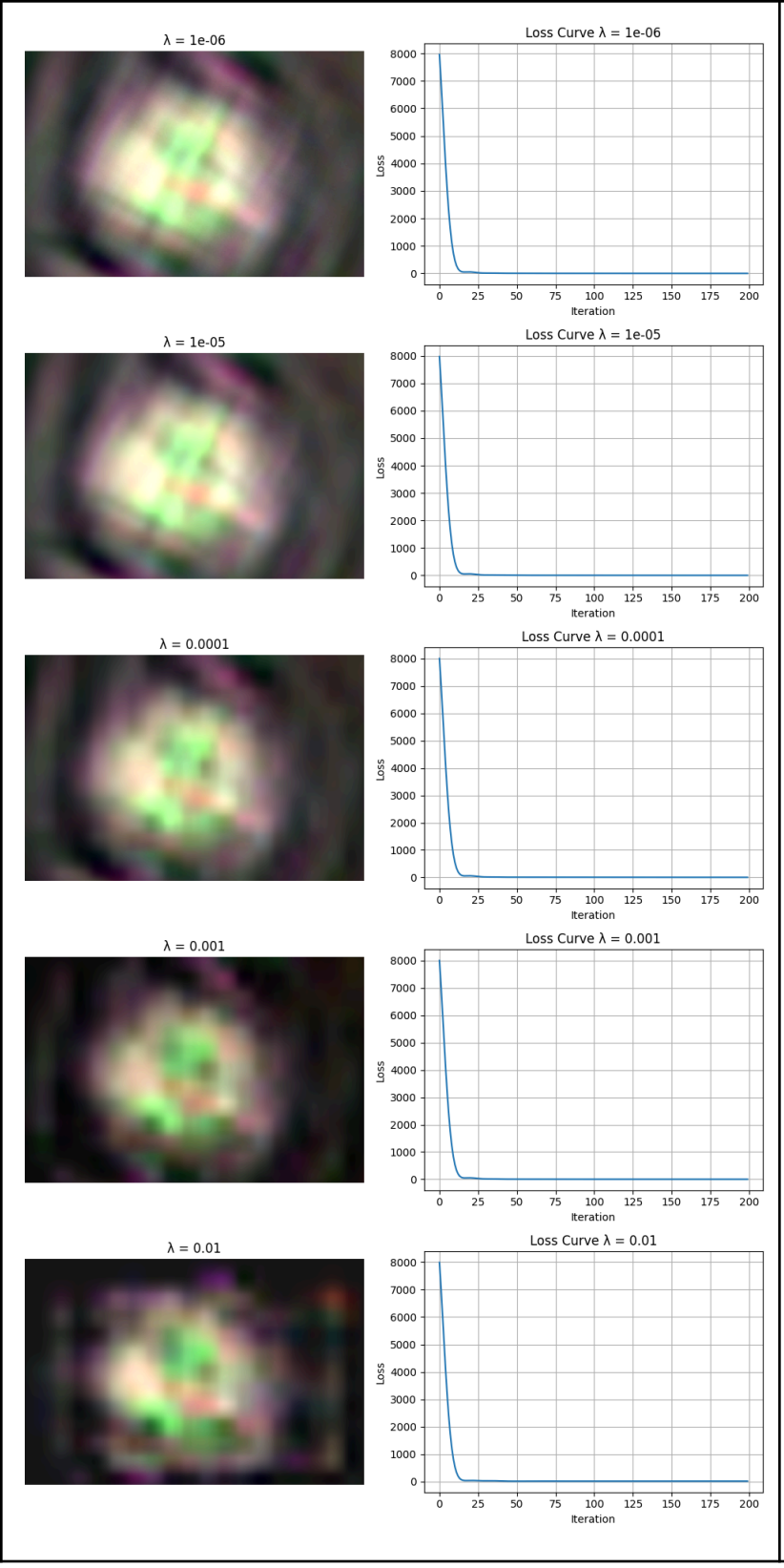
b)

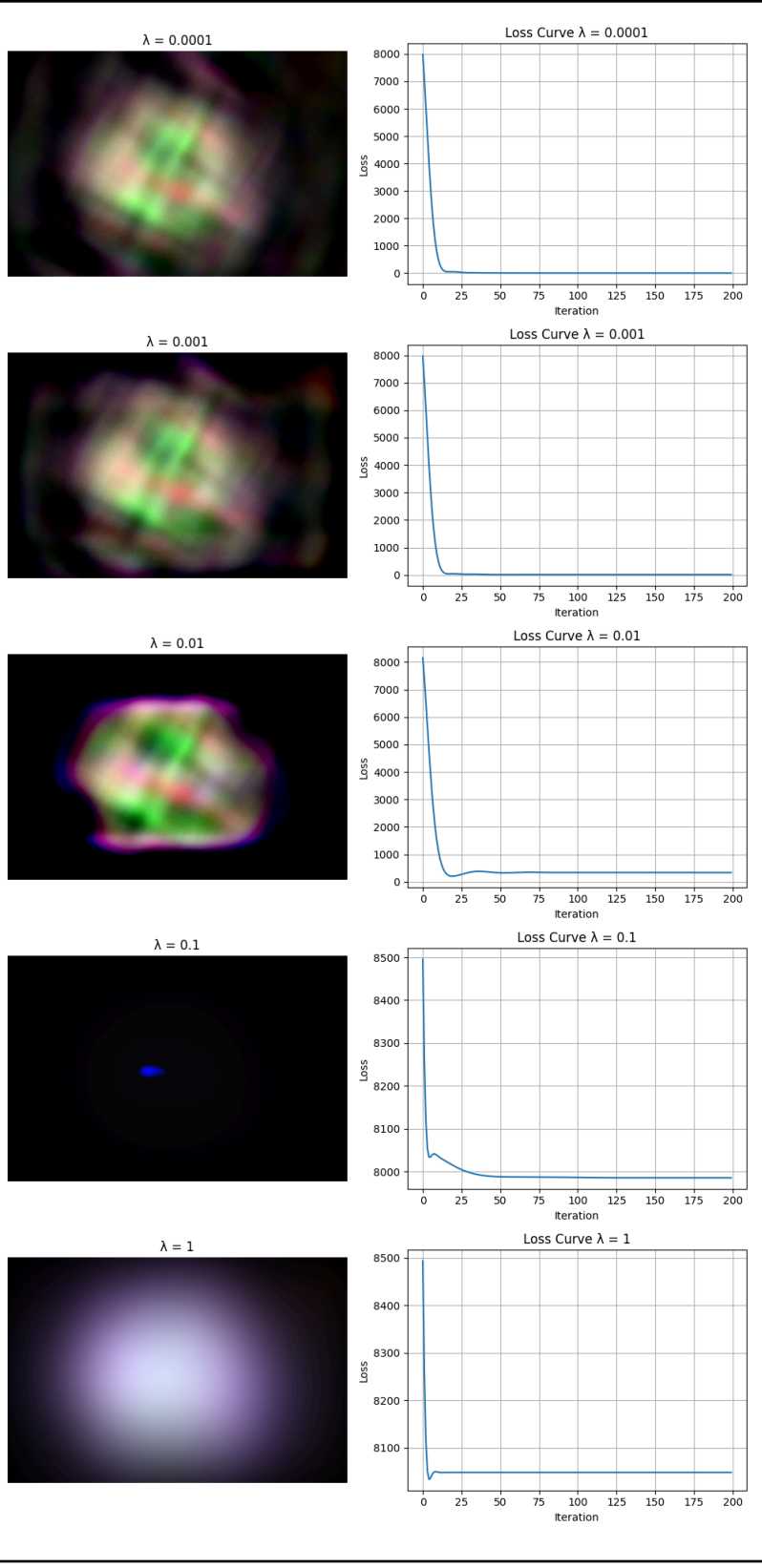
Real Image	Reconstructed Image
	

Nonzero coefficients: 364

You could keep the top 0.2% of the coefficients and still maintain a PSNR of greater than 22dB.

c)





d) As Lambda increases, the reconstruction becomes less noisy, but also you begin to lose some of the finer details. Overall, it appears that the wavelet result is slightly better at preserving detail compared to the naive result.

2 Total Variation

Another common image prior is total variation (TV), which promotes sparse image gradients. Most natural images consist of smooth regions separated by sharp boundaries. The gradient $\nabla \mathbf{x}$ of such an image is zero almost everywhere except at edges, making it sparse. The TV prior exploits this structure: by penalizing the ℓ_1 norm of the gradient, it encourages solutions with flat regions and sharp transitions between them.

The loss function with a TV prior becomes:

$$L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|\nabla \mathbf{x}\|_1$$

The proximal operator for TV does not have a closed-form solution, which makes it complicated to use, and often requires a nested iteration within the optimization loop. Luckily, there exists an approximation to the TV prior using Haar wavelets that requires no nested loops.

The function `tv3dApproxHaar(x,tau)` is already provided in `priors.py`. It uses the Haar transform helpers (`ht3`, `iht3`) to approximate the TV proximal operator. You will use it to implement `prox_tv`, which combines non-negativity and TV via the proximal average:

$$\text{prox}(\mathbf{x}) = \frac{1}{2}\left(\text{prox}_{\text{nonneg}}(\mathbf{x}) + \text{prox}_{\text{TV}}(\mathbf{x})\right)$$

This is the same technique used in `prox_native` from Lab 8, where non-negativity was averaged with ℓ_1 soft-thresholding.

Add the total variation prior to your FISTA code and try running your image using the TV prior. You will need to tune λ until you see an effect. Setting the TV prior too high will result in an image that's blocky and cartoon-like. Setting the TV prior too low will result in an image that looks identical to one that has no prior. You should tune λ until artifacts and noise are suppressed, resulting in a sharp image that's not too cartoon-like. Make sure to plot the loss ($L(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda\|\nabla \mathbf{x}\|_1$) over iteration.

Problem 2.1:

(a) Fill in `prox_tv` in `priors.py`. It should combine non-negativity and TV via the proximal average, using the provided `tv3dApproxHaar`.

(b) Run FISTA with `prox_fn=prox_tv`, sweeping λ (e.g. 10^{-6} to 10^{-2}) on your DiffuserCam image for 200 iterations. Display reconstructions.

(c) What happens as λ increases? Compare TV reconstructions to your wavelet results from Problem 1.1.

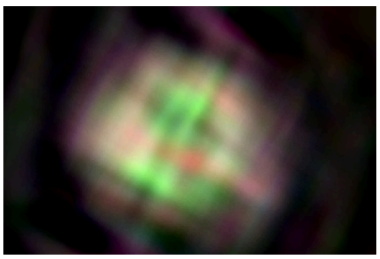
Problem 2.2: Compare FISTA with all four priors (non-negativity, native sparsity, wavelet, TV) on your DiffuserCam image. Tune each λ for best quality. Plot final images and loss curves side by side. Which prior performs best for your image, and why?

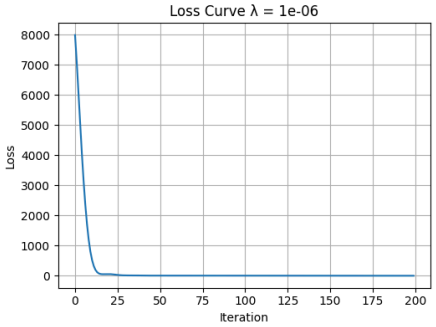
a) Done

b) Done

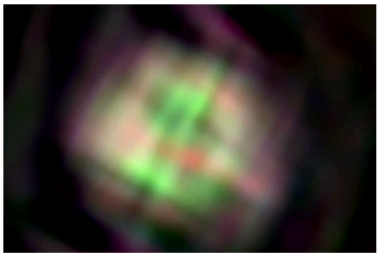
c) As lambda increases, it will suppress the noise, while keeping it looking “natural” by preserving edges (but as it becomes extreme, it will still keep the general structure but just be blurry). Compared to wavelets, TV produces sharper edges, but looks generally worse because it will lose out on the texture.

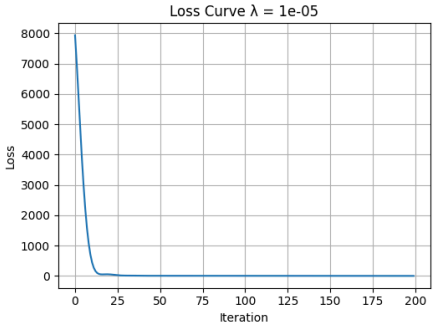
λ = 1e-06



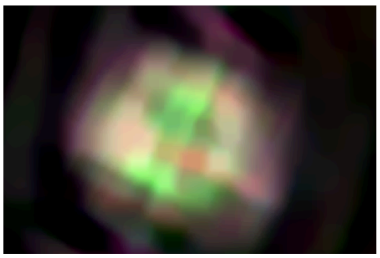


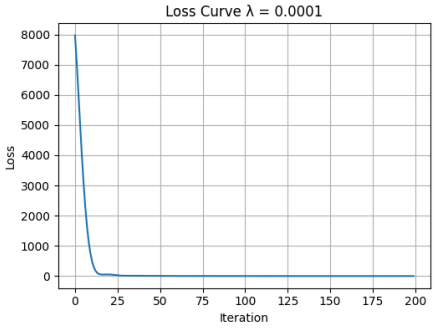
λ = 1e-05



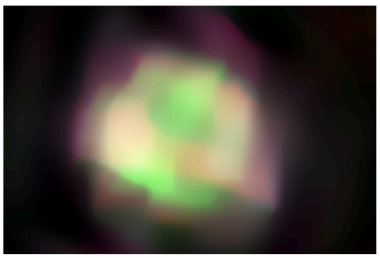


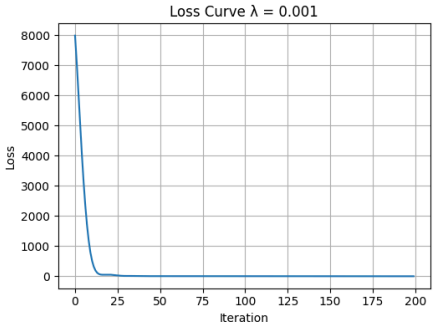
λ = 0.0001



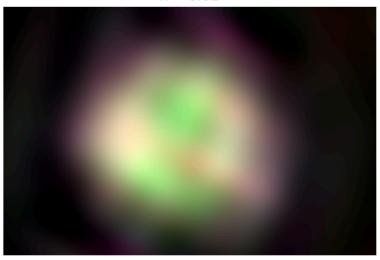


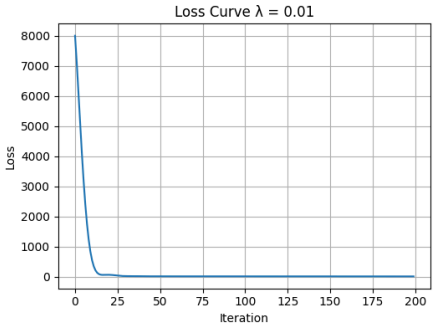
λ = 0.001



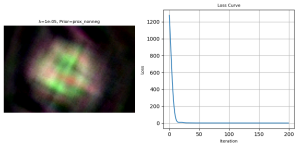
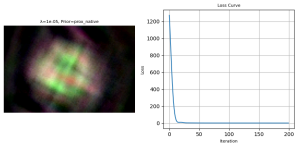
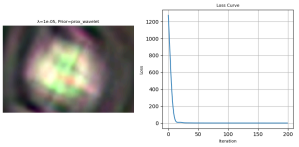
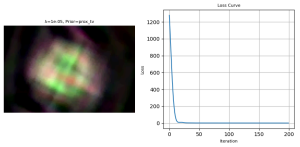
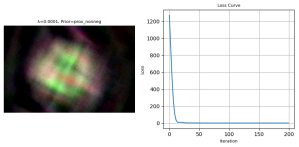
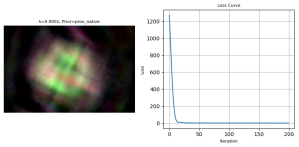
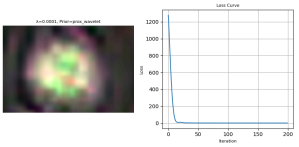
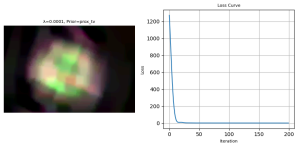
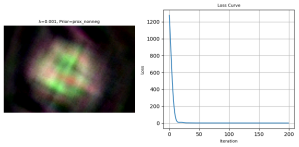
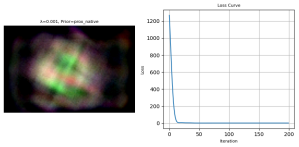
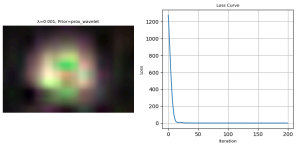
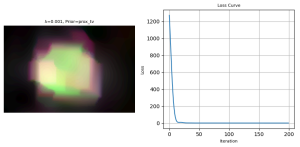
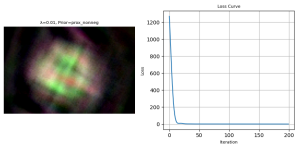
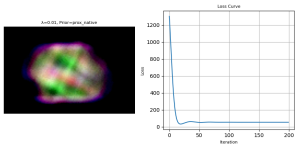
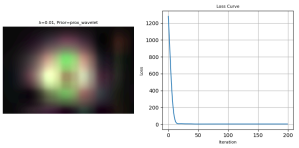
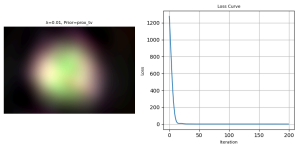


λ = 0.01





2.2)

	prox_nonneg	prox_native	prox_wavlet	prox_tv
$\lambda = 0.00001$				
$\lambda = 0.0001$				
$\lambda = 0.001$				
$\lambda = 0.01$				

Based on the images, the $\lambda = 0.00001$ prox_tv appeared to give the most clear result.

3 Mutual Coherence

In compressed sensing, we want to recover a signal from fewer measurements than unknowns. Whether this is possible depends on how "different" the measurement system is from the sparsity basis. Mutual coherence quantifies this.

For a matrix A with columns $\mathbf{a}_1, \ldots, \mathbf{a}_n$, the mutual coherence is:

$$\mu(A) = \max_{i \neq j} \frac{|\langle \mathbf{a}_i, \mathbf{a}_j \rangle|}{\|\mathbf{a}_i\| \cdot \|\mathbf{a}_j\|}$$

Mutual coherence ranges from 0 to 1. A low value means no two columns of the matrix look alike, which is ideal for compressed sensing—it means each measurement captures genuinely different information about the signal. A high value means some columns are nearly redundant.

The cross-coherence between two matrices A and B measures how similar their columns are:

$$\mu(A, B) = \max_{i,j} \frac{|\langle \mathbf{a}_i, \mathbf{b}_j \rangle|}{\|\mathbf{a}_i\| \cdot \|\mathbf{b}_j\|}$$

The following helper function is provided. `dct_matrix(n)` constructs the $n \times n$ orthonormal DCT-II matrix.

```
def dct_matrix(n):
    D = np.zeros((n, n))
    for k in range(n):
        for i in range(n):
            D[k, i] = np.cos(
                np.pi * k * (2*i + 1) / (2*n))
    D[0, :] *= 1 / np.sqrt(n)
    D[1:, :] *= np.sqrt(2 / n)
    return D
```

Problem 3.1:

(a) Implement `mutual_coherence(A)` that computes $\mu(A)$ as defined above. It should iterate over all pairs of columns, normalize each, and return the maximum absolute inner product.

(b) Implement `cross_coherence(A,B)` that computes $\mu(A, B)$. It should iterate over all pairs of one column from A and one column from B.

(c) Generate a wide random Gaussian matrix `A=np.random.randn(32,64)` and a DCT matrix of the same shape: `D=dct_matrix(64)[:32,:]`. Compute the self-coherence of each using `mutual_coherence`, and the cross-coherence using `cross_coherence(A,D)`. Which matrix has lower self-coherence, and why? How does the cross-coherence compare?

(d) Create a matrix with near-duplicate columns: `A_noisy=A+0.1*np.random.randn(32,64)`. Compute `cross_coherence(A,A_noisy)`. What happens to the coherence? Why would high coherence between a measurement matrix and a sparsifying basis be bad for compressed sensing?

a) Done

b) Done

c)

Mutual Coherence of A: 0.5795255905404952
Mutual Coherence of D: 0.6701408134272816
Cross Coherence between A and D: 0.5488702593595665

Mutual Coherence is lower for matrix A compared to matrix D. This is because random Gaussian matrix columns are more likely to be orthogonal to each other, leading to a smaller inner product. In contrast, a truncated DCT matrix is not nearly as orthogonal, as it has rows that are removed from the normally orthogonal matrix, which leads to a higher coherence. The cross-coherence between A and D is lower, as there is not a strong correlation between a random matrix and a truncated DCT matrix.

d)

Cross Coherence between A and A_noisy: 0.9978220504104308

The cross coherence between A and A_noisy is very high, since the columns of A and A_noisy are almost identical. High coherence is problematic for compressive sensing as it means that the measurements being made are not providing independent information. If the measurement matrix is highly coherent, then it would struggle to distinguish between different sparse representations, leading to a poor image recovery.

4 Compressive Imaging

Up until now, we've been solving inverse problems that have not been underdetermined. For 2D imaging with DiffuserCam, we have N measurements and N unknowns. Now, we'll try to solve an underdetermined inverse problem, where we have fewer measurements than unknowns. This is the same principle that can be used to recover 3D and hyperspectral images from 2D measurements.

Compressed sensing theory tells us that recovery is possible when two conditions hold: (1) the signal is sparse in some basis, and (2) the measurement matrix is incoherent with that basis—as you explored in the previous section. DiffuserCam satisfies the second condition naturally: each scene point spreads across many sensor pixels, creating a multiplexed measurement that distributes information redundantly. This is why DiffuserCam can tolerate missing measurements better than a conventional lens, where each pixel captures only one scene point.

You will try two different strategies to compress your measurement:

1. Masking out a random fraction (e.g. 90%, 95%, 99%) of the pixels in your measurement.
2. Masking out a rectangle of pixels in the center of your field of view. You can adjust the size of this rectangle to mask out different fractions of total pixels.

In each case, you will form a binary mask, M, with zeros in regions that should be masked out and 1s in regions of the measurement to keep. You will then element-wise multiply this binary mask with your measurement to form your compressed measurement, $\mathbf{y}_{\text{compressed}} = M \cdot \mathbf{y}$.

Use FISTA to recover an image from your compressed measurement. You will need to update your `A` and `A_adjoint` methods in `inverse_solver.py` to support masking. Pass the mask to `InverseSolver` via the `mask` argument:

```
# Example: random mask keeping 50% of pixels
mask_flat = np.ones(np.prod(y.shape))
n_zero = int(len(mask_flat) * 0.5)
mask_flat[:n_zero] = 0
np.random.shuffle(mask_flat)
mask = mask_flat.reshape(y.shape)
```

The forward model becomes $\mathbf{A} = M\,\text{crop}(\mathbf{x} * \mathbf{h})$, and the adjoint needs the corresponding change: multiply $\mathbf{y}$ by M before padding. You can verify your masked adjoint using the adjoint test.

We recommend using FISTA with a TV prior to solve the inverse problem, but a wavelet prior could also work. Make sure to tune λ until you get the best reconstructed image quality.

Problem 4.1:

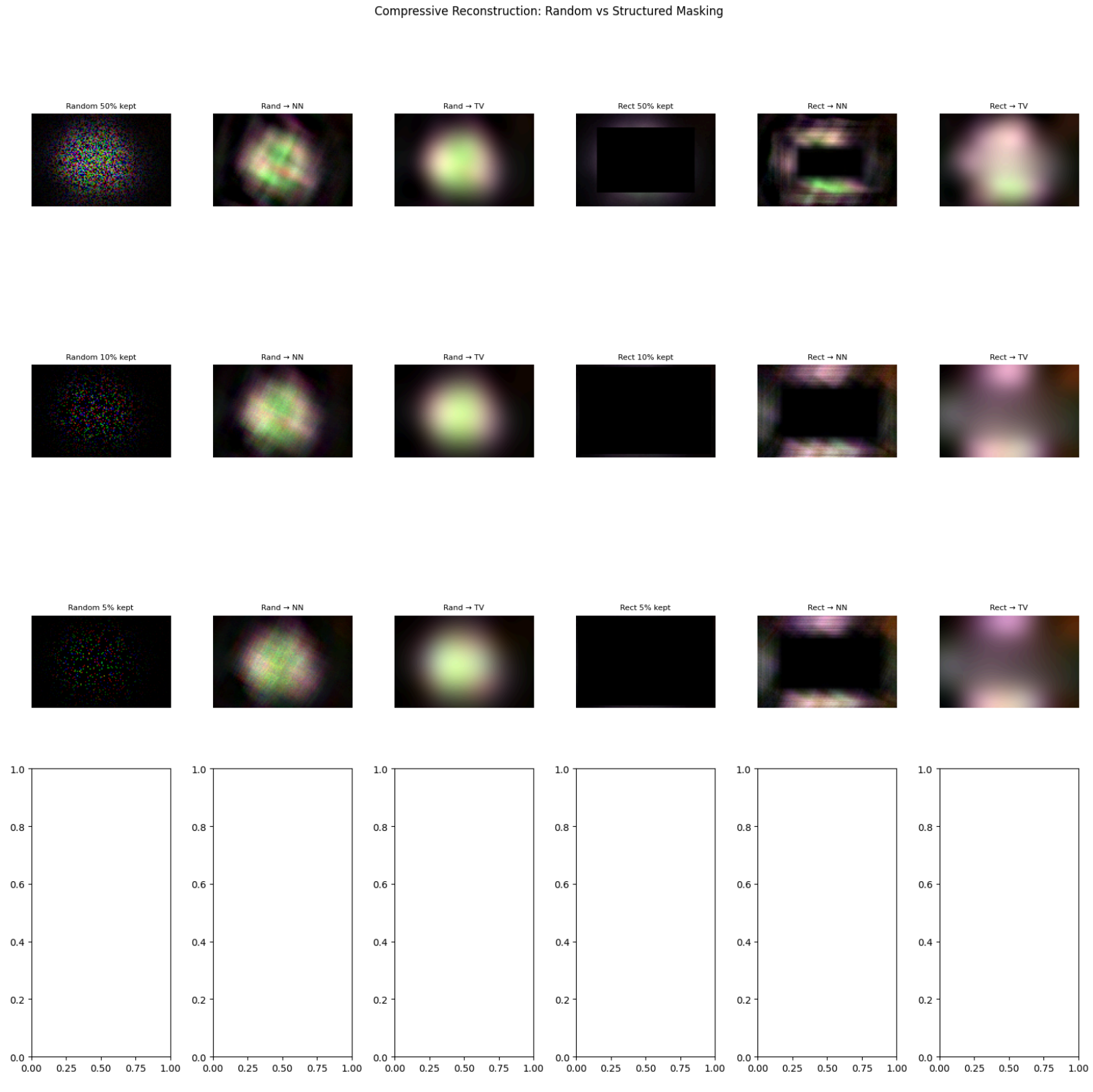
(a) Update `A` and `A_adjoint` in `inverse_solver.py` to support `self.mask` (if you haven't already). For `A`: multiply the output by `self.mask` after cropping. For `A_adjoint`: multiply `y` by `self.mask` before padding. Verify your masked forward and adjoint operators with the adjoint test: generate random $\mathbf{x}$ and $\mathbf{y}$, and check that $\langle \mathbf{Ax}, \mathbf{y} \rangle \approx \langle \mathbf{x}, \mathbf{A}^*\mathbf{y} \rangle$. If the adjoint test fails, your reconstructions in part (b) will not converge correctly.

(b) Reconstruct your DiffuserCam image under random masking (keeping e.g. 50%, 10%, 5% of pixels) and structured masking (center rectangle removed). Display the masked measurements and reconstructions compared to no masking. Compare reconstruction quality using TV prior vs non-negativity only—why does the choice of prior matter more for compressive measurements?

a) Done

b) The TV reconstruction looks slightly better, because it encodes an actual constraint on the pixels rather than just naive non-negativity, and this matters more in compressive

sensing since the measurements aren’t enough to really determine a unique solution, the prior matters more to enforce.



Submission

Submit the following on Gradescope:

- A single PDF with answers to all problems (Sections 1–4), including all requested figures, code, and discussion. Submit this to "Lab 9 Writeup."
- Your code files (`inverse_solver.py`, `priors.py`, and any scripts). Include a `README.md` explaining how to run your code. Submit this to "Lab 9 Code."